

TABELAS DERIVADAS E CTE

Tiago Henrique Angioletti





O QUE SÃO TABELAS DERIVADAS?

DEFINIÇÃO: Uma tabela derivada é uma subconsulta usada na cláusula FROM que age como uma tabela temporária durante a execução da consulta principal.

FUNCIONALIDADE: Permite criar uma “tabela intermediária” em tempo de execução, sem a necessidade de criar uma tabela física no banco de dados.



SINTAXE DE TABELAS DERIVADAS

```
SELECT coluna1, coluna2
FROM (
    SELECT coluna1, coluna2
    FROM tabela
    WHERE condição
) AS tabela_derivada
WHERE outra_condição;
```

EXPLICAÇÃO:

A subconsulta é colocada entre parênteses e age como uma “tabela”.

A tabela derivada recebe um nome temporário (tabela_derivada) que pode ser referenciado na consulta principal.



EXEMPLO PRÁTICO DE TABELA DERIVADA

```
SELECT cliente_id, valor_venda
FROM (
    SELECT cliente_id, AVG(valor) AS valor_venda
    FROM vendas
    GROUP BY cliente_id
) AS vendas_media
WHERE valor_venda > 500;
```

EXPLICAÇÃO DO EXEMPLO:

A subconsulta calcula o valor médio das vendas (valor_venda) por cliente e o armazena na tabela derivada vendas_media.

A consulta principal seleciona clientes onde o valor médio das vendas é superior a 500.



VANTAGENS DO USO DE TABELAS DERIVADAS

SIMPLIFICAÇÃO DE CONSULTAS COMPLEXAS: Tabelas derivadas tornam consultas complexas mais organizadas ao isolar partes específicas de cálculos.

EFICIÊNCIA: Em alguns casos, as tabelas derivadas evitam a criação de tabelas físicas temporárias, economizando recursos.

FLEXIBILIDADE: Como a tabela derivada é definida na própria consulta, não é necessário criar objetos adicionais no banco de dados.



LIMITAÇÕES DAS TABELAS DERIVADAS

NÃO REUTILIZÁVEIS: Diferente das CTEs, que podem ser referenciadas várias vezes em uma consulta, tabelas derivadas só podem ser usadas uma vez e não são reutilizáveis.

DESEMPENHO: Podem impactar o desempenho quando a subconsulta é complexa ou contém muitos dados, já que a tabela derivada é calculada em cada execução da consulta.

ORGANIZAÇÃO: Para consultas muito grandes, o uso de tabelas derivadas pode reduzir a legibilidade, especialmente se houver várias camadas de subconsultas aninhadas.

INTRODUÇÃO A CTE E AO COMANDO **WITH**



O QUE SÃO CTES?

Common Table Expressions são consultas temporárias referenciadas dentro de uma instrução SQL.

Simplificam consultas complexas, tornando-as mais legíveis e fáceis de manter.

POR QUE USAR **WITH** EM VEZ DE SUBCONSULTAS ANINHADAS?

Melhora a legibilidade e evita repetição.

CASOS RECOMENDADOS PARA O USO DE CTES

Consultas complexas, cálculos intermediários e subconsultas reutilizáveis.



SINTAXE DO COMANDO WITH

ESTRUTURA BÁSICA

```
WITH nome_cte AS (  
    SELECT ...  
)  
SELECT * FROM nome_cte;
```

NOME_CTE: nome da CTE, usada como uma tabela temporária.

SELECT * FROM NOME_CTE: consulta principal usando a CTE.



CRIANDO UMA CTE SIMPLES

```
WITH vendas_totais AS (  
    SELECT cliente_id, SUM(valor) AS total_vendas  
    FROM vendas  
    GROUP BY cliente_id  
)  
SELECT * FROM vendas_totais;
```

EXPLICAÇÃO

`vendas_totais` calcula o total de vendas por cliente.

A consulta principal exibe o total de vendas por cliente.



UTILIZANDO MÚLTIPLAS CTES

EXEMPLO COM MÚLTIPLAS CTES

```
WITH vendas_totais AS (  
    SELECT cliente_id, SUM(valor) AS total_vendas  
    FROM vendas  
    GROUP BY cliente_id  
) , clientes_vip AS (  
    SELECT cliente_id  
    FROM vendas_totais  
    WHERE total_vendas > 10000  
)  
SELECT * FROM clientes_vip;
```

EXPLICAÇÃO

`clientes_vip` usa o resultado de `vendas_totais` para filtrar clientes com total acima de 10.000.



CONSULTAS RECURSIVAS COM CTE

CTE RECURSIVA PARA HIERARQUIA DE FUNCIONÁRIOS

```
WITH RECURSIVE hierarquia AS (  
  SELECT id, nome, chefe_id, 1 AS nivel  
  FROM funcionarios  
  WHERE chefe_id IS NULL  
  UNION ALL  
  SELECT f.id, f.nome, f.chefe_id, h.nivel + 1  
  FROM funcionarios f  
  INNER JOIN hierarquia h ON f.chefe_id = h.id  
)  
SELECT * FROM hierarquia;
```

EXPLICAÇÃO

`nivel` aumenta à medida que a hierarquia é percorrida.

Útil para estruturar dados hierárquicos.



VANTAGENS DE USAR CTES

LEGIBILIDADE E ORGANIZAÇÃO: facilita a leitura e manutenção.

REDUÇÃO DE SUBCONSULTAS ANINHADAS: CTEs substituem consultas redundantes.

REUTILIZAÇÃO: cálculos intermediários podem ser usados em várias partes da consulta.



LIMITAÇÕES E CUIDADOS AO USAR CTES

IMPACTO NA PERFORMANCE: CTEs podem ser lentas para grandes volumes de dados.

VERSÕES DO MYSQL: CTEs só estão disponíveis a partir do MySQL 8.0.



EXERCÍCIO PRÁTICO

Criar uma CTE para calcular o total de vendas por cliente e listar clientes com vendas acima de uma meta.

```
WITH vendas_totais AS (  
    SELECT cliente_id, SUM(valor) AS total_vendas  
    FROM vendas  
    GROUP BY cliente_id  
)  
SELECT cliente_id, total_vendas  
FROM vendas_totais  
WHERE total_vendas > 5000;
```

Usar uma CTE recursiva para gerar uma lista hierárquica de departamentos.